

```

log.error("Filter 처리 중 Exception 발생: {}", e.getMessage(), e);
response.sendError(HttpStatus.BAD_REQUEST, "Filter 처리 중 Exception 발생.");
return;
} finally {
    if (ServiceContextHolder.getInstance() != null) {
        ServiceContextHolder.removeInstance();
    }
    if (!ThreadContext.isEmpty()) {
        ThreadContext.clearAll();
    }
}

private <T> T convertMapToDto(ObjectMapper mapper, Map<String, Object> map, Class<T> clazz) {
    map.remove("dtoLogicalName");
    return mapper.convertValue(map, clazz);
}

package nhnis.fw.commons.filter;

import java.io.BufferedReader;
import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;
import jakarta.servlet.ReadListener;
import jakarta.servlet.ServletInputStream;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletRequestWrapper;

public class CachedBodyHttpServletRequest extends HttpServletRequestWrapper {
    private final byte[] cachedBody;

    public CachedBodyHttpServletRequest(HttpServletRequest request) throws IOException {
        super(request);
        this.cachedBody = request.getInputStream().readAllBytes();
    }

    public byte[] getCachedBody() {
        return cachedBody;
    }

    @Override
    public ServletInputStream getInputStream() {
        ByteArrayInputStream inputStream = new ByteArrayInputStream(cachedBody);
        return new ServletInputStream() {
            @Override
            public boolean isFinished() {
                return inputStream.available() == 0;
            }

            @Override
            public boolean isReady() {
                return true;
            }
        }
    }
}

```

```

@Override
public void setReadListener(ReadListener arg0) {
}

@Override
public int read() throws IOException {
    return inputStream.read();
}

@Override
public BufferedReader getReader() throws IOException {
    return new BufferedReader(new InputStreamReader(getInputStream()),
        StandardCharsets.UTF_8);
}
}

```

```

package nhnis.fw.commons.filter;

import java.io.IOException;
import java.util.Enumeration;
import java.util.Map;
import java.util.stream.Collectors;

import org.apache.logging.log4j.ThreadContext;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpHeaders;
import org.springframework.http.client.reactive.ClientHttpConnector;
import org.springframework.stereotype.Component;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.databind.ObjectMapper;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.FilterConfig;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.extern.slf4j.Slf4j;
import nhnis.fw.commons.context.ServiceContextHolder;
import nhnis.fw.commons.dto.header.sys_comm;
import nhnis.fw.commons.dto.header.hdr_nhnis;
import nhnis.fw.commons.jwt.JwtProvider;

@Component
@Slf4j
public class DefaultFilter implements Filter {

    private final ClientHttpConnector connector;

    private static final String GUID = "guid";
    private static final String IP = "ip";
}

```

```

private static final String USER_ID
private static final String SERVICE_ID
private static final String HEADER
private static final String MULTIPART
= "userId";
= "serviceId";
= "hdr_nhnis";
= "multipart/";

@Value("${spring.profiles.active:local}")
private String active;
@Value("${spring.application.name}")
private String applicationName;

@Autowired
private JwtProvider jwtProvider;

DefaultFilter(ClientHttpConnector connector) {
    this.connector = connector;
}

@Override
public void init(FilterConfig filterConfig) {
    if (log.isDebugEnabled()) {
        log.debug("Application DefaultFilter is loaded");
    }
}

@Override
}

@Override
public void doFilter(ServletRequest servletRequest, ServletResponse servletResponse, FilterChain
    filterChain) throws IOException, ServletException {
    HttpServletRequest request = (HttpServletRequest) servletRequest;
    HttpServletResponse response = (HttpServletResponse) servletResponse;

    // ThreadContext All
    String uri = request.getRequestURI();
    String serviceId = uri.contains("/") ? uri.substring(uri.lastIndexOf("/") + 1) : "UNKNOWN";
    ThreadContext.put(SERVICE_ID, serviceId);

    HttpHeaders requestHeaders = new HttpHeaders();
    Enumeration<String> names = request.getHeaderNames();
    while (names.hasMoreElements()) {
        String name = names.nextElement();
        requestHeaders.add(name, request.getHeader(name));
    }

    try {
        if (request.getContentType() != null) {
            if (request.getContentType().startsWith(MULTIPART)) {
                String guid = request.getHeader("X-GUID");
                sys_comm sysComm = new sys_comm();
                sysComm.setStd_gbl_id(guid);
                hdr_nhnis header = new hdr_nhnis();
                header.setSys_comm(sysComm);
                ThreadContext.put(GUID, guid);
                ServiceContext serviceContext = new ServiceContext(applicationName,
                    guid, active, requestHeaders, request, response, header);
                ServiceContextHolder.setInstance(serviceContext);
                filterChain.doFilter(request, servletResponse);
            } else {
                CachedBodyHttpRequest wrapper =
                    new CachedBodyHttpRequest(request);
            }
        }
    }
}

```

```

        ThreadContext.put(GUID, header.getSys_comm().getStd_gbl_id());

        String guid = header.getSys_comm().getStd_gbl_id();

        hdr_nhnis = header = convertMapToDto(mapper, headerMap,
    }

    Header 정보가 없습니다.");

    if (headerMap == null) {
        response.sendError(HttpServletResponse.SC_BAD_REQUEST, "유효
rootMap.getHeader(HEADER);

    Map<String, Object> headerMap = (Map<String, Object>)
    }

    return;

    된 JSON 형식입니다.");

    } catch (JsonProcessingException e) {
        log.error("Request Body JSON 파싱 실패 : {}", e.getMessage(), e);
        response.sendError(HttpServletResponse.SC_BAD_REQUEST, "잘못
TypeReference<Map<String, Object>>()) {
        rootMap = mapper.readValue(requestBody, new
    try {
        Map<String, Object> rootMap = null;

        mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);

        ObjectMapper mapper = new ObjectMapper();

        return;

        "Request Body 입력 전문이 비어있습니다.");
        response.sendError(HttpServletResponse.SC_BAD_REQUEST,
    if (requestBody == null || requestBody.isEmpty()) {

        String requestBody = wrapper.getReader().lines().collect(Collectors.joining(System.lineSeparator()));

        }

        String ssold = jwtProvider.getSold(token);
        request.setAttribute("ssold", ssold);

        return;

        response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Access Token이 아닙니다.");

        } else if (!jwtProvider.isAccessToken(token)) {
            return;

        response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "유효하지 않은 Token입니다.");

        if (!jwtProvider.validate(token)) {

        String token = authorization.substring(7);

        }

        response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Access Token이 없습니다.");

        return;

        if (authorization == null || !authorization.startsWith("Bearer ")) {

        request.getHeader(HttpHeaders.AUTHORIZATION);
        String authorization =
            if (inactive.toLowerCase().equals("local")) {
                authorization
            }
        }
    }

```